

The Intelligent Bistro — Interview Q&A Prep Guide

A structured question bank with in-depth answers based on the actual codebase. Use this to rehearse explaining **what** the app does, **why** it's built this way, and **how** each piece works.

Part 1: Project Overview & Motivation

Q1. What is The Intelligent Bistro, and what problem does it solve?

Answer:

The Intelligent Bistro is a restaurant ordering app built for the Viridien AI Full-Stack Engineering internship challenge. Guests browse a menu, manage a cart, place orders, and interact with an AI maître d' that understands natural language.

The core problem is not "chat with a bot" — it's **reliable cart mutations from messy human language**. Phrases like "add 4 sandwiches and 7 burgers with 3 lemonades" or "remove the small fries and change my burger to large" must become **structured, testable actions** (`ADD` , `REMOVE` , `SET_MODIFIER`) that the UI can apply deterministically.

The app demonstrates:

- Cross-platform mobile (Expo/React Native)
 - A stateless Express API
 - A hybrid AI pipeline (rules + OpenAI)
 - Voice ordering on web and native
-

Q2. What are the main user-facing features?

Answer:

Four tabs, implemented in `mobile/app/(tabs)/` :

Tab	Feature
Menu	Browse ~30 items across 7 categories, filter, pull-to-refresh, add to cart with size/modifiers
AI Assistant	Conversational ordering, menu Q&A, recommendations, voice input
Cart	Edit quantities, change sizes, 8% tax, manual checkout
Orders	Session order history, cancel orders

The AI can also place orders, cancel orders, answer menu questions, suggest pairings, and handle multi-turn confirmations ("Place order" → "yes").

Q3. Why did you choose this tech stack?

Answer:

Mobile — Expo SDK 54 + React Native 0.81

- One codebase for iOS, Android, and web
- Expo Router for file-based navigation
- Fast iteration with Expo Go on a physical device

State — Zustand

- Lightweight global state for cart, orders, and menu cache
- No Redux boilerplate for a demo-scale app

Styling — NativeWind (Tailwind)

- Consistent dark "bistro" theme with utility classes

Backend — Express + TypeScript (Node 22)

- Simple REST API, easy to reason about
- ESM modules, Zod validation, `tsx` for dev hot reload

AI — OpenAI (gpt-4o + Whisper)

- Strong NLU for cart parsing
- Whisper for voice on Expo Go where browser speech APIs aren't available

No database — intentional for MVP speed; menu is in-memory TypeScript, cart/orders live on the client.

Q4. Is this a monorepo? How is the project organized?

Answer:

Yes, informally. Two sibling packages with **no root** `package.json` or workspace tooling:

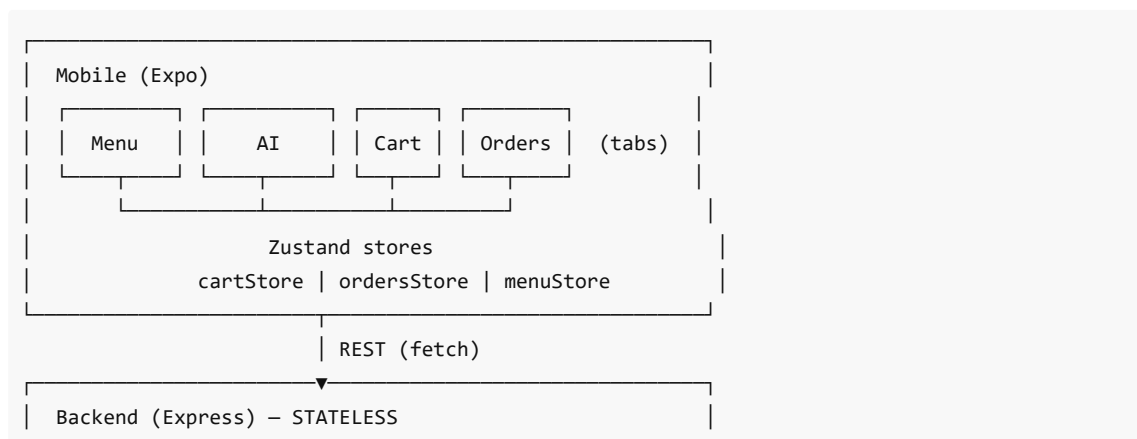
```
viridien_project_intelligent_bistro/  
├─ backend/      # Express API  
├─ mobile/       # Expo app  
├─ docker-compose.yml  
├─ README.md  
└─ docs/        # Session notes
```

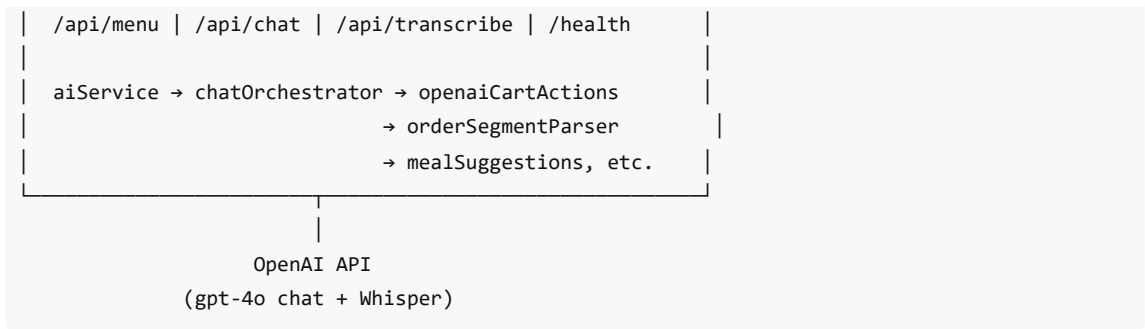
Each package installs and runs independently. Docker only containerizes the API; the mobile app always runs locally via Metro/Expo Go.

Part 2: System Architecture

Q5. Walk me through the high-level architecture.

Answer:





Key principle: The backend is a **pure function** — it receives message + cart snapshot + order snapshot + session context, and returns reply + structured actions. It never stores cart or order state.

Q6. Why is the backend stateless? What are the tradeoffs?

Answer:

Why stateless:

- Horizontal scaling: any API instance can handle any request
- Simpler deployment: no Redis/DB session layer for the demo
- Clear separation: **client owns truth** for cart and orders

How it works: Every `POST /api/chat` includes:

- `cart` : current lines + subtotal
- `orders` : placed order snapshots
- `session` : awaitingConfirmation (`place_order` | `bulk_add`) and `pendingActions`

The server returns `actions` , `orderActions` , `sessionContext` , and sometimes `placeOrderFromCart: true` .

Tradeoffs:

Pro	Con
Easy to scale	Client must send full snapshots every message
No session store	State lost on app restart
Testable pure functions	Multi-device sync impossible without a DB

For production, I'd add Redis or server-side sessions with user accounts.

Q7. Explain the "structured actions" pattern. Why not let the LLM directly mutate state?

Answer:

The LLM returns **commands**, not mutations:

```
// CartAction types
"ADD" | "REMOVE" | "UPDATE_QUANTITY" | "SET_MODIFIER" | "CLEAR"

// OrderAction types
"CANCEL_ORDER" | "CANCEL_ALL_ORDERS"
```

The mobile client applies them via `cartStore.applyActions()` and `ordersStore.applyOrderActions()` .

Benefits:

- 1. **Predictability** — same action types from rules or AI
- 2. **Testability** — golden tests assert action arrays, not prose
- 3. **UI sync** — cart updates immediately and consistently
- 4. **Auditability** — `parsedBy` field shows `"openai"` , `"rules"` , `"openai+rules"` , etc.

Free-text-only replies would require fragile parsing on the client and invite hallucinated cart changes.

Q8. What API endpoints exist and what does each do?

Answer:

Method	Path	Purpose
GET	/health	Liveness; reports ai: "openai" "rules" and voice: "whisper" "rules-only"
GET	/api/menu	Full menu with modifiers
GET	/api/menu/categories	Category list
POST	/api/chat	NL → reply + CartAction[] + optional OrderAction[] + session
POST	/api/transcribe	Base64 audio → Whisper transcription

Entry point: `backend/src/index.ts` — CORS, 16MB JSON limit (for audio), route mounting.

Part 3: AI & NLP Pipeline (Critical Interview Area)

Q9. How does a chat message get processed end-to-end?

Answer:

Flow in `aiService.ts` → `chatOrchestrator.ts` :

```
POST /api/chat
|
├ 1. Greeting? ("hello") → getGreetingReply() with featured dishes
|
├ 2. handleStructuredChat() – rules-first orchestration
|   ├── Yes/no confirmations (place_order, bulk_add)
|   ├── Cancel / list / detail orders
|   ├── "Place order" → cart summary + await "yes"
|   ├── Compound: menu question + cart add in one message
|   ├── Menu inquiry ("what are your starters?")
|   └─ Cart mutations → resolveCartActions()
|       ├── OPENAI_API_KEY set? → openaiCartActions + reconcile
|       └─ else → orderSegmentParser (pure rules)
|
└ 3. If still unresolved && OpenAI key → general JSON chat (temp 0.2)
    └ on failure → rulesFallback()
```

After cart mutations, `mealSuggestions.ts` may add pairing advice and recommendation blocks.

Q10. Why rules-first *and* OpenAI? Isn't AI enough?

Answer:

Each layer has a job:

Rules (`chatOrchestrator` , `orderSegmentParser`) — deterministic flows:

- Confirmations ("yes" / "no")
- Place order workflow
- Bulk quantity guard (>10 items)
- Order cancel/list
- Menu category listing

OpenAI (`openaiCartActions.ts`) — flexible NLU:

- Complex multi-item orders
- Typos, filler words, casual phrasing
- Size modifiers on natural language

Design choice: OpenAI is an **enhancement**, not a requirement. Without `OPENAI_API_KEY`, the app still works via rules. `/health` reports which mode is active.

This gives reliability for critical flows and flexibility for messy language.

Q11. What is `reconcileAiCartActions` and why did you need it?

Answer:

This is a strong "hard problem I solved" story.

Bug: When the client sent the full cart snapshot with a message like "add lemonade," the LLM sometimes **re-added every item already in the cart** because it treated the cart context as instructions.

Fix in `orderSegmentParser.ts` :

```
export function reconcileAiCartActions(message: string, aiActions: CartAction[]):
CartAction[] {
  const rules = parseRulesCartActions(message);
  const rulesMutations = rules.filter(a => ADD | REMOVE | CLEAR);

  if (rulesMutations.length > 0) {
    // Intersect: only allow AI ADD/REMOVE that rules also detected
    // Prefer AI's version (better modifiers) when keys match
  }

  // Fallback: filter AI ADDs to items actually mentioned in message
  const filtered = aiActions.filter(a =>
    a.type !== "ADD" || itemMentionedInMessage(message, a.itemId)
  );
}
```

Two strategies:

1. If rules detect mutations → merge with AI (AI wins on matching keys for better size/modifier parsing)
2. If rules find nothing → strip AI `ADD` actions for items not mentioned in the current message

Validated by golden tests in `validate-order-parser.ts` (19 cases including hallucination scenarios).

Q12. How does the dedicated OpenAI cart parser differ from the general chat fallback?

Answer:

`openaiCartActions.ts` is a **focused, low-temperature** parser:

- **Input:** user message + read-only cart context (for disambiguation, not re-adding)
- **Output:** `{"actions": [...]}` only — no conversational reply
- **Temperature:** 0 for determinism
- **Post-processing:** default `size: "medium"` on ADDs, then `reconcileAiCartActions()`

The general fallback in `aiService.ts` handles broader chat (menu Q&A mixed with cart) at temperature 0.2, with a large system prompt embedding the full menu catalog.

Separation keeps cart parsing testable and reduces prompt complexity per task.

Q13. How do you handle compound messages like "What are your bowls? Also add 2 soup"?

Answer:

`handleCompoundMessage()` in `chatOrchestrator.ts` :

1. `normalizeCompoundMessage()` splits menu inquiry from cart intent
2. Menu part → `menuInquiry.ts` / `mealSuggestions.ts`
3. Cart part → `resolveCartActions()`
4. Reply combines both: category listing + cart change summary

`messageNormalizer.ts` detects patterns like "also add", "and add", "plus" to separate intents.

Q14. How does the place-order confirmation flow work?

Answer:

Multi-turn, client-held session:

1. User: "Place my order"
2. Server: cart summary with tax, reply "Reply yes to confirm", sets `sessionContext.awaitingConfirmation: "place_order"`
3. Client stores session in `assistant.tsx` state
4. User: "yes" → server returns `placeOrderFromCart: true`
5. Client calls `ordersStore.placeOrderFromCart()` — snapshots cart, assigns order number (from 1001), clears cart

Server never places the order itself; it signals the client. Same pattern for **bulk_add** when quantity > `HIGH_QUANTITY_THRESHOLD` (10).

Q15. How does menu matching work for NLP?

Answer:

`backend/src/data/menu.ts` defines ~30 items with:

- `id` (e.g. `sparkling-water`)
- `name`, `category`, `price`
- `aliases` (e.g. `"burger"`, `"caesar"`)
- `modifiers` injected via `withStandardSizeModifiers()`

`orderSegmentParser.ts` uses:

- Alias/name matching
- Word-number parsing (`"three"` → 3)
- `expandWithClauses()` for `"with 3 lemonades"` patterns
- Size extraction via `sizeParser.ts`

Every `itemId` in actions is validated against `MENU_ITEMS`.

Q16. What are recommendation blocks and suggestion chips?

Answer:

Rich UI elements in `ChatResponse`:

- `suggestionChips` — tappable phrases (`"Add truffle fries"`) sent as the next message
- `recommendationBlocks` — structured picks with emoji, price, note, and `addMessage`

Generated by `mealSuggestions.ts` after cart adds — e.g. missing drink/side/dessert gaps via `analyzeMealGaps()`, plus pairing suggestions.

Keeps the assistant proactive without hardcoding replies in the mobile app.

Part 4: Frontend (Mobile)

Q17. How is navigation structured?

Answer:

Expo Router file-based routing:

```
mobile/app/
├─ _layout.tsx      # Root: loads menu, SafeArea, gesture handler
└─ (tabs)/
  ├─ _layout.tsx    # Tab bar (Menu, AI, Cart, Orders)
  ├─ index.tsx      # Menu
  ├─ assistant.tsx  # AI chat
  ├─ cart.tsx
  └─ orders.tsx
```

`(tabs)` is a route group — shared tab layout without affecting the URL path.

Q18. Explain the Zustand stores and their responsibilities.

Answer:

Store	File	Responsibility
menuStore	menuStore.ts	Fetches and caches menu from API on app load
cartStore	cartStore.ts	Cart lines, applyActions(), subtotal, merge logic
ordersStore	ordersStore.ts	Placed orders, placeOrderFromCart(), applyOrderActions()

Cart line identity: Lines merge when same `itemId` + identical modifiers (`lineKey`). Each line gets a unique `lineId` for UI operations.

`applyActions()` handles all `CartAction` types the AI returns — the bridge between server intelligence and UI state.

Q19. How does the AI Assistant screen wire up the chat flow?

Answer:

`assistant.tsx` :

1. On mount, sends "hello" for greeting + starter chips
2. On send, calls `sendChatMessage()` with message, history, cart snapshot, order snapshots, session
3. Applies response:
 - `actions` → `applyActions(actions, menuItems)`
 - `orderActions` → `applyOrderActions()`
 - `placeOrderFromCart` → `placeOrderFromCart()`
 - `sessionContext` → `setSession()`
4. Updates message history and composer chips from response

Cart and orders are read from Zustand at send time — always fresh snapshots.

Q20. How does platform-aware API URL resolution work?

Answer:

`mobile/src/lib/api.ts` :

Platform	URL
Web	<code>apiUrlLocal</code> → <code>http://localhost:3001</code>
Android emulator	<code>http://10.0.2.2:3001</code> (host loopback)
Physical device	<code>apiUrlDevice</code> → LAN IP (e.g. <code>http://172.x.x.x:3001</code>)
iOS Simulator	<code>apiUrlLocal</code>

Configured in `app.json` → `expo.extra` . Physical devices can't use `localhost` — they need the host machine's Wi-Fi IP. README documents firewall troubleshooting on Windows.

Q21. How is pricing calculated, including modifiers?

Answer:

`mobile/src/lib/menuModifiers.ts` (mirrored in backend `data/menuModifiers.ts`):

- Every item gets a `size` modifier: small/medium/large with category-based `priceDelta`
- `computeUnitPrice(item, modifiers)` = base price + sum of modifier deltas
- Default size: **medium** when unspecified

Cart subtotal = $\sum (\text{unitPrice} \times \text{quantity})$. Tax is **8%**, computed on client in cart/orders stores and in server summaries for place-order confirmation.

Q22. How does the Menu screen fetch and display data?

Answer:

`index.tsx` (Menu tab):

- `menuStore` loads via `GET /api/menu` in root `_layout.tsx` on app start
 - Category filter via `CategoryFilter` component
 - `MenuItemCard` + `SizeSelector` + `AddToCartButton`
 - Pull-to-refresh re-fetches menu
 - Error banner shows failed API URL for debugging connectivity
-

Part 5: Voice Input

Q23. How does voice input work across platforms?

Answer:

Platform-specific hooks via Expo's `.web.ts` / `.native.ts` resolution:

Web (`useVoiceInput.web.ts`):

- Browser Web Speech API
- Live partial + final transcripts
- Mic requires **localhost or HTTPS** (browser security)
- Auto-restart on silence, retry on empty results

Native (`useVoiceInput.native.ts`):

- **Expo Go:** record with `expo-av` → `POST /api/transcribe` (Whisper)
- **Dev/production build:** `expo-speech-recognition` for live native speech when available

Detection: `Constants.executionEnvironment === "storeClient"` means Expo Go → Whisper path.

Q24. Why Whisper on Expo Go instead of always using native speech?

Answer:

Expo Go is a sandbox — native speech modules aren't fully available. Whisper via the API works everywhere you have `OPENAI_API_KEY` and network.

Tradeoff: higher latency (record → upload → transcribe) vs instant web speech. Acceptable for demo; production builds get live recognition.

Q25. Why is the JSON body limit 16MB?

Answer:

`express.json({ limit: "16mb" })` in `index.ts` — voice recordings sent as base64 to `/api/transcribe` can be large. Default Express limit (~100KB) would reject them.

Part 6: Data Models

Q26. Describe the core data types and how they relate.

Answer:

```
MenuItem (server catalog, static TS array)
  |
  | referenced by itemId
  ▼
CartLine (client) —placeOrderFromCart()→ Order (client)
  ▲
  | applyActions(CartAction[])
  |
ChatResponse.actions from API
```

MenuItem: id, name, description, category, price, image, tags, modifiers, aliases

CartLine: lineId, itemId, name, quantity, unitPrice, modifiers

Order: id, orderNumber (starts 1001), status (`placed` | `cancelled`), lines snapshot, subtotal, tax, total, timestamps

No ORM — string `itemId` references validated at runtime against `MENU_ITEMS` .

Q27. What is `ChatSessionContext` ?

Answer:

```
{
  awaitingConfirmation?: "place_order" | "bulk_add" | null,
  pendingActions?: CartAction[] // held during bulk qty confirm
}
```

Client stores this in React state (`assistant.tsx`), sends it with each message, server returns updated `sessionContext` . Not persisted server-side — lightweight multi-turn state without a session store.

Q28. What does `parsedBy` tell you?

Answer:

Telemetry/debug field on `ChatResponse` :

- `"rules"` — pure rule-based parsing
- `"openai"` — OpenAI general fallback
- `"rules-multi"` — multiple rule segments
- `"openai+rules"` — OpenAI cart parser after reconcile

Useful for monitoring which path handled a message in production.

Part 7: Backend Implementation Details

Q29. How is request validation handled?

Answer:

Zod schemas at the route layer. Example: `chat.ts` validates message (1–2000 chars), history, cart lines, orders, session. Invalid body → `400` with `error.flatten()`.

`aiService.ts` also validates OpenAI JSON responses with Zod before trusting them.

Q30. How is the menu served? Could you change it without redeploying?

Answer:

Currently: static array in `backend/src/data/menu.ts`, served by `routes/menu.ts`. Changes require code edit + redeploy.

Production path: PostgreSQL or headless CMS, admin UI, image CDN. Menu route becomes a DB query with caching.

Q31. What is `ruleBasedParser.ts`? Is it used?

Answer:

Legacy file — **not imported anywhere**. Superseded by `orderSegmentParser.ts`. Good to mention if asked about dead code: kept during refactor, safe to delete.

Q32. How does order cancellation work via chat?

Answer:

`orderParser.ts` detects cancel intent → returns `OrderAction: { type: "CANCEL_ORDER" }` (last order) or `CANCEL_ALL_ORDERS`.

Client `ordersStore.applyOrderActions()` marks orders `cancelled` with `cancelledAt`. User can also cancel from the Orders tab UI.

Part 8: DevOps, Config & Deployment

Q33. How do you run the app locally?

Answer:

```
# Backend
cd backend && npm install && npm run dev # port 3001

# Mobile
cd mobile && npm install && npx expo start

# Or API in Docker
docker compose up --build -d
```

Phone on same Wi-Fi → set `apiUrlDevice` in `mobile/app.json` to host LAN IP.

Q34. Explain the Docker setup.

Answer:

`docker-compose.yml` builds `backend/Dockerfile` (multi-stage Node 22):

- Exposes port 3001
- Loads `backend/.env` via `env_file` (important: don't override `OPENAI_*` in compose `environment` or keys get wiped)
- Healthcheck hits `/health` every 30s
- `restart: unless-stopped`

Mobile is **not** containerized — always runs on host via Metro.

Q35. What environment variables matter?

Answer:

Backend (`backend/.env`):

Variable	Default	Purpose
PORT	3001	HTTP port
OPENAI_API_KEY	—	Enables OpenAI + Whisper
OPENAI_MODEL	gpt-4o	Chat + cart parsing

Mobile (`app.json` **extra**):

- `apiUrlLocal` , `apiUrlDevice`
-

Q36. How would you deploy this to production?

Answer:

Documented but not implemented:

API: Fly.io, Railway, Render, AWS ECS, or Azure Container Apps — Docker image + env secrets + HTTPS

Mobile: EAS Build → TestFlight / Play Store internal testing

Additions needed:

- Auth (JWT/OAuth)
 - PostgreSQL for menu + orders
 - Redis for sessions
 - Rate limiting on `/api/chat` and `/api/transcribe`
 - Lock down CORS (currently `origin: true`)
 - Observability (latency, `parsedBy` , error rates)
 - Queue for AI calls under load
-

Part 9: Testing & Quality

Q37. How is the codebase tested?

Answer:

No Jest/Vitest. **Custom golden-test scripts:**

Script	Cases	Covers
<code>npm run validate:parser</code>	19	Cart parsing, chains, with-clauses, reconcile anti-hallucination
<code>npm run validate:ai</code>	23	Greeting, menu inquiry, place order, cancel, compound messages

Mobile: `npx tsc --noEmit` type-check only.

Recommended CI gate:

```
cd backend && npm run build && npm run validate:parser && npm run validate:ai
cd mobile && npx tsc --noEmit
```

Philosophy: test **action outputs**, not LLM prose — stable across model updates.

Q38. Why golden tests instead of mocking OpenAI?

Answer:

- Rules path is fully deterministic — always testable without API key
- OpenAI cart parser tests can run live when key is set
- Reconcile layer tests simulate AI hallucination input → assert filtered output
- Avoids brittle mock setups for JSON schema drift

For CI without API key, rules + reconcile tests still pass.

Part 10: Security, Limitations & Honest Tradeoffs

Q39. What security measures exist today?

Answer:

Minimal — appropriate for internship demo, not production:

- Zod input validation (message length, typed bodies)
- No secrets in client code (API key server-only)
- No auth — API is open
- Permissive CORS (`origin: true`)
- No rate limiting

Production: API keys in secrets manager, auth, rate limits, HTTPS, CORS allowlist, input sanitization, audit logs.

Q40. What are the biggest limitations of the current design?

Answer:

Limitation	Impact
Client-side cart/orders	Lost on app restart
No user accounts	No order history across devices
In-memory menu	Deploy required for menu changes
Sync OpenAI per message	Latency + cost at scale
Tax 8% duplicated	Client + server both compute (could drift)
Order numbers reset	nextOrderNumber resets on app restart
No payments	Orders are simulated

Being explicit about these shows production awareness.

Q41. What was the hardest bug you fixed?

Answer:

Cart context hallucination: Sending full cart snapshot caused the LLM to re-add all items on a simple "add lemonade" request.

Fix stack:

1. Dedicated cart parser with explicit "do not re-add cart items" prompt rules
2. `reconcileAiCartActions()` — intersect with rules, filter unmentioned ADDs
3. Golden test: AI returns full cart → reconcile strips to only lemonade

Also challenging: **"with 3 lemonades"** attaching quantity to the wrong item — solved with `expandWithClauses()` and prompt rules.

Q42. Why 8% tax and where is it applied?

Answer:

`TAX_RATE = 0.08` in `chatOrchestrator.ts` for place-order summaries; same rate in `cartStore / ordersStore` on client. Hardcoded business rule for demo — production would use configurable tax by jurisdiction.

Part 11: Scalability & Future Architecture

Q43. How would you scale this to 10,000 concurrent users?

Answer:

Layer	Today	Scale path
API	Single Express instance	Horizontal pods behind load balancer
AI	Sync OpenAI per request	Queue (Bull/SQS) + worker pool, streaming responses
State	Client snapshots	Redis sessions + server cart for logged-in users

Menu	Static TS	PostgreSQL + CDN + cache
Voice	Whisper per request	Batch or dedicated speech service
Observability	Console logs	Datadog/Sentry, parsedBy metrics

Stateless design helps API scaling; AI cost and latency become the bottleneck first.

Q44. How would you add payments?

Answer:

Stripe or Apple Pay after `placeOrderFromCart()` :

1. Server creates `PaymentIntent` with order total
2. Client confirms payment
3. Webhook marks order `paid`
4. Kitchen display receives order via `WebSocket`

Cart/checkout stays client-side until payment succeeds, then persist order server-side.

Q45. How would you add a kitchen display system?

Answer:

`WebSocket` or SSE from API when order is placed:

- `ordersStore.placeOrderFromCart()` also `POST /api/orders`
- Server broadcasts to kitchen clients
- Order states: `placed` → `preparing` → `ready`

Requires moving orders server-side with persistence.

Part 12: Behavioral & Design Questions

Q46. Why Zustand over Redux or React Context?

Answer:

Small global state (cart, orders, menu cache), few actions, no middleware needs. Zustand is ~1KB, no providers, simple `applyActions` API. Redux would add boilerplate without benefit at this scale. Context would cause unnecessary re-renders for cart updates across tabs.

Q47. Why Expo over bare React Native?

Answer:

Faster setup, Expo Go for instant device testing, unified web support, Expo Router included. Tradeoff: some native modules need dev builds (speech recognition) — acceptable for internship timeline.

Q48. Why TypeScript on both sides?

Answer:

Shared mental model for `CartAction` , `MenuItem` , `ChatResponse` . Types in `backend/src/types` and `mobile/src/types` mirror each other. Catches contract mismatches at compile time — critical when API returns structured JSON the client must apply.

Q49. If you had another week, what would you prioritize?

Answer:

- 1. Server-side order persistence (PostgreSQL)
- 2. E2E tests (Detox/Maestro) for place-order flow
- 3. Rate limiting + basic auth for API
- 4. EAS production build with native speech
- 5. Delete `ruleBasedParser.ts` , extract shared types to a package

Shows prioritization judgment.

Q50. How does this project demonstrate full-stack AI engineering?

Answer:

- **Full-stack:** Expo client + Express API + Docker + cross-platform concerns
 - **AI engineering:** Not just "call GPT" — structured outputs, reconcile layer, rules fallback, dedicated parsers, golden tests
 - **Product thinking:** Confirmations, bulk guards, recommendations, voice on multiple platforms
 - **Production mindset:** Health checks, validation, extensive README, documented tradeoffs
-

Quick "Explain in 60 Seconds" Script

"The Intelligent Bistro is an AI-powered restaurant ordering app. Users browse a menu, chat with a maître d', and manage orders — on iOS, Android, or web via Expo. The backend is a stateless Express API: every chat request includes the cart snapshot, and the server returns structured actions like `ADD` or `REMOVE`, not just text. We use a hybrid pipeline — rules for confirmations and order flow, OpenAI for messy natural language, plus a reconcile layer that prevents the LLM from hallucinating cart changes. Cart and orders live in Zustand on the client; there's no database in the MVP. Voice works via Web Speech on web and Whisper on Expo Go. It's Dockerized for the API and validated with 42 golden test cases."

Files to Know Cold (Open These Before the Interview)

File	One-liner
<code>backend/src/index.ts</code>	API entry, routes, health
<code>backend/src/services/aiService.ts</code>	Chat orchestration entry
<code>backend/src/services/chatOrchestrator.ts</code>	Structured flows, confirmations
<code>backend/src/services/openaiCartActions.ts</code>	OpenAI cart parser
<code>backend/src/services/orderSegmentParser.ts</code>	Rules NLP + reconcile
<code>backend/src/data/menu.ts</code>	Catalog source of truth
<code>mobile/src/store/cartStore.ts</code>	Cart + applyActions

mobile/app/(tabs)/assistant.tsx	AI UX + session
mobile/src/lib/api.ts	Platform API URLs

Generated for Viridien interview preparation — The Intelligent Bistro codebase.